

Hierarchical Markov Modeling for Underwater Pursuit-Evasion

1 Overview

This document provides three consistent formulations based on the project implementation:

- Low-level 6DOF control MDP.
- High-level pursuit-evasion task MDP.
- Macro-action (Options) SMDP for hierarchical control.

2 Low-Level 6DOF Control MDP

We model the low-level controller as an MDP:

$$M_{LL} = (\mathcal{S}_{LL}, \mathcal{A}_{LL}, P_{LL}, R_{LL}, \gamma). \quad (1)$$

State

The state is the full 6DOF vehicle state plus goal:

$$s_t^{LL} = (\eta_t, \nu_t, g_t), \quad (2)$$

where $\eta_t = [x, y, z, \phi, \theta, \psi]^\top$ is position and orientation, $\nu_t = [u, v, w, p, q, r]^\top$ is linear and angular velocity, and g_t denotes the target state (position/orientation, and optionally target velocity).

Action

The low-level action space is discrete with 13 semantic actions:

$$\mathcal{A}_{LL} = \{a_0, a_1, \dots, a_{12}\}. \quad (3)$$

They correspond to hover, surge/sway/heave, and yaw/pitch/roll in positive or negative directions.

Transition

The action is mapped to thrust through a control stack, then the 6DOF dynamics evolve:

$$\tau_t = f_{ctrl}(a_t, s_t^{LL}), \quad (4)$$

$$s_{t+1}^{LL} = f_{dyn}(s_t^{LL}, \tau_t, \xi_t), \quad (5)$$

where ξ_t denotes disturbances (water current, domain randomization).

Reward

The reward combines position/orientation error, velocity penalty, energy cost, and goal/time terms:

$$r_t^{LL} = w_p \|e_p\| + w_o \|e_o\| + w_v \|\nu\| + w_e \|\tau\| + r_{goal} + r_{time}. \quad (6)$$

Termination

Episodes terminate on success (within thresholds), out-of-bounds, flip, no-progress, or timeout.

3 High-Level Pursuit-Evasion MDP

The pursuit-evasion task is modeled as:

$$M_H = (\mathcal{S}_H, \mathcal{A}_H, P_H, R_H, \gamma_H). \quad (7)$$

State

A task-level state can be defined by relative geometry and boundary information:

$$s_t^H = [p_p, v_p, p_e, v_e, \psi_e, b_p, b_e], \quad (8)$$

where p_p, v_p are pursuer position and velocity, p_e, v_e are evader position and velocity, ψ_e is evader heading, and b_p, b_e represent boundary margins.

Action

High-level actions represent sub-goal selection or pursuit strategies:

$$\mathcal{A}_H = \{\text{subgoal/heading/strategy}\}. \quad (9)$$

Transition

High-level actions are realized by running the low-level policy for multiple steps:

$$s_{t+1}^H = F(s_t^H, a_t^H, \pi_{LL}). \quad (10)$$

Reward

A typical high-level reward for pursuit-evasion:

$$r_t^H = \alpha \Delta d + \beta v_{close} + r_{catch} + r_{boundary} - r_{time}. \quad (11)$$

Termination

Capture, failure (escape/out-of-bounds), or timeout.

4 Macro-Action / Options SMDP

The hierarchical control can be modeled as an SMDP with options:

$$M_S = (\mathcal{S}, \mathcal{O}, P_S, R_S, \gamma). \quad (12)$$

Options

Each option is defined as:

$$o = (\mathcal{I}_o, \pi_o, \beta_o), \quad (13)$$

where $\mathcal{I}_o$ is the initiation set, π_o is the intra-option policy (low-level controller), and β_o is the termination condition.

Transition

An option runs for k steps before transitioning:

$$P_S(s' \mid s, o) = \sum_{k \geq 1} P(s', k \mid s, o). \quad (14)$$

Return

The SMDP reward accumulates over the option duration:

$$R_S(s, o) = \mathbb{E} \left[\sum_{i=0}^{k-1} \gamma^i r_{t+i} \right]. \quad (15)$$

5 Observation Vector Definitions

This section enumerates the exact observation vectors used in the code.

5.1 2D Pursuit-Evasion Observation (22D)

Let $\Delta p = p_e - p_p$, $\Delta v = v_e - v_p$, $d = \|\Delta p\|$, and

$$v_{close} = -\frac{\Delta v \cdot \Delta p}{\|\Delta p\|} \quad (d > 0). \quad (16)$$

Normalization constants:

$$\text{pos_scale} = \frac{1}{\text{world_size}}, \quad \text{vel_scale} = \frac{1}{\max(v_{p_max}, v_{e_max})}, \quad \text{acc_scale} = \frac{1}{\max(a_{p_max}, a_{e_max})}. \quad (17)$$

The 22-dimensional observation is ordered as:

$$\begin{aligned}
1 &: p_{p,x} \cdot \text{pos_scale}, & 2 &: p_{p,y} \cdot \text{pos_scale}, \\
3 &: v_{p,x} \cdot \text{vel_scale}, & 4 &: v_{p,y} \cdot \text{vel_scale}, \\
5 &: \Delta p_x \cdot \text{pos_scale}, & 6 &: \Delta p_y \cdot \text{pos_scale}, \\
7 &: \Delta v_x \cdot \text{vel_scale}, & 8 &: \Delta v_y \cdot \text{vel_scale}, \\
9 &: d \cdot \text{pos_scale}, & 10 &: v_{close} \cdot \text{vel_scale}, \\
11 &: \hat{v}_{e,x}, & 12 &: \hat{v}_{e,y}, \\
13 &: (\text{world_size} - p_{p,y}) \cdot \text{pos_scale}, \\
14 &: (p_{p,y} + \text{world_size}) \cdot \text{pos_scale}, \\
15 &: (p_{p,x} + \text{world_size}) \cdot \text{pos_scale}, \\
16 &: (\text{world_size} - p_{p,x}) \cdot \text{pos_scale}, \\
17 &: \min(\text{world_size} - |p_{e,x}|, \text{world_size} - |p_{e,y}|) \cdot \text{pos_scale}, \\
18 &: \|v_p\| \cdot \text{vel_scale}, & 19 &: \|v_e\| \cdot \text{vel_scale}, \\
20 &: \hat{v}_p \cdot \hat{\Delta p}, \\
21 &: a_{p,x} \cdot \text{acc_scale}, & 22 &: a_{p,y} \cdot \text{acc_scale}.
\end{aligned}$$

5.2 Low-Level 6DOF Observation

Let $R(\phi, \theta, \psi)$ be the rotation matrix from body to world. The body-frame position error is:

$$e_p^{body} = R(\phi, \theta, \psi)^\top (g_p - \eta_{pos}). \quad (18)$$

The orientation error is the wrapped angle difference:

$$e_o = \text{wrap}(\eta_{ori} - g_{ori}) \in [-\pi, \pi]^3. \quad (19)$$

The observation is assembled in segments depending on configuration flags. Each segment is appended in the following order:

- Position error (body frame): $e_p^{body}/10$.
- Orientation error: e_o/π .
- Linear velocity (body frame): $[u, v, w]/2$.
- Angular velocity (body frame): $[p, q, r]/2$.
- Target position (body frame): $e_p^{body}/10$.
- Target orientation: g_{ori}/π .
- Error terms: $e_p^{body}/10$ and e_o/π .

The final observation is the concatenation of all enabled segments in that order.

6 How to Compile

From the project root, run:

```
cd docs
pdflatex markov_modeling.tex
```